

# Restaurant Profitability Prediction

## & Customer Segmentation using Machine Learning

Zomato Dataset — End-to-End ML System with Streamlit Deployment

| Field           | Details                                                                      |
|-----------------|------------------------------------------------------------------------------|
| Project Title   | Restaurant Profitability Prediction & Customer Segmentation                  |
| Dataset         | Zomato Bangalore Restaurants   51,717 records   17 columns                   |
| Problem Type    | Supervised Regression + Unsupervised Clustering                              |
| Target Variable | Restaurant Rating (1.0-5.0) → Profitability Score (0-100)                    |
| Best Model      | Random Forest   $R^2 = 0.5071$   $MAE = 0.5034$                              |
| Clusters Found  | 6 segments identified via K-Means + Silhouette Score                         |
| Models Compared | Linear Regression, Ridge, Decision Tree, Random Forest, Gradient Boosting    |
| Deployment      | Streamlit Web App (multi-page, interactive predictor, dark theme)            |
| Libraries       | pandas · numpy · scikit-learn · matplotlib · seaborn · streamlit · reportlab |

## TABLE OF CONTENTS

| #  | Section                             |
|----|-------------------------------------|
| 1  | Problem Statement & Objectives      |
| 2  | Dataset Description & Features      |
| 3  | Data Preprocessing Pipeline         |
| 4  | Exploratory Data Analysis (EDA)     |
| 5  | Profitability Scoring Framework     |
| 6  | Baseline Supervised Learning        |
| 7  | Model Evaluation & Metrics          |
| 8  | Overfitting / Underfitting Analysis |
| 9  | Model Optimization (GridSearchCV)   |
| 10 | K-Means Clustering (Unsupervised)   |
| 11 | Tree-Based Models                   |
| 12 | Model Comparison & Final Selection  |
| 13 | Complete ML Pipeline Summary        |
| 14 | Streamlit Web Application           |
| 15 | Conclusions & Future Scope          |
| 16 | References                          |

# 1 PROBLEM STATEMENT & OBJECTIVES

## 1.1 Background

The food delivery industry generates millions of data points daily. Platforms like Zomato host thousands of restaurants but lack a systematic way to assess which restaurants are likely to perform well financially. Manual review is impossible at scale. This project addresses that gap using machine learning.

## 1.2 Problem Definition

**Primary:** Predict a restaurant's aggregate rating (1.0–5.0) using features such as location, cuisine, cost, and service availability — ratings being the strongest proxy for customer satisfaction and repeat business.

**Secondary:** Convert the predicted rating into a composite Profitability Score (0–100) by combining it with popularity, pricing, and service signals.

**Tertiary:** Segment all restaurants into business clusters using unsupervised learning to enable targeted operational strategies.

## 1.3 Objectives

- Implement a clean, reproducible ML pipeline from raw data to deployment.
- Train and compare 5 regression models (Linear, Ridge, Decision Tree, Random Forest, Gradient Boosting).
- Apply hyperparameter tuning via GridSearchCV with 5-fold cross-validation.
- Detect and mitigate overfitting using regularisation and ensemble methods.
- Perform K-Means clustering with Elbow + Silhouette analysis to find optimal segments.
- Build an interactive Streamlit web application for live profitability prediction.

| Dimension          | Value                                           |
|--------------------|-------------------------------------------------|
| Problem Type       | Supervised Regression + Unsupervised Clustering |
| Target Variable    | rating (continuous, 1.0–5.0)                    |
| Business Output    | Profitability Score (0–100) + Business Tier     |
| Evaluation Metrics | MAE, RMSE, $R^2$ (train & test)                 |
| Dataset            | Zomato Bangalore Restaurants — 51,717 rows      |

## 2 DATASET DESCRIPTION & FEATURES

### 2.1 Source & Overview

The Zomato Bangalore Restaurants dataset was sourced from Kaggle. It contains restaurant listing data for Bangalore, India — 1,500 raw records across 17 columns. After preprocessing, the working dataset contains 1,500 records and 29 engineered features.

### 2.2 Raw Column Mapping

| Original Column             | Renamed To      | Data Type | Role    | Notes                                   |
|-----------------------------|-----------------|-----------|---------|-----------------------------------------|
| rate                        | rating          | Float     | TARGET  | Cleaned from '4.1/5', 'NEW', '-' format |
| approx_cost(for_two_people) | cost_for_two    | Integer   | Feature | Stripped commas: '1,200' → 1200         |
| votes                       | votes           | Integer   | Feature | Number of user ratings                  |
| online_order                | online_order    | Binary    | Feature | Yes/No → 1/0                            |
| book_table                  | book_table      | Binary    | Feature | Yes/No → 1/0                            |
| rest_type                   | restaurant_type | Cat       | Feature | OHE applied (top-15 kept)               |
| cuisines                    | cuisines        | Cat       | Feature | OHE applied (top-15 kept)               |
| listed_in(city)             | city            | Cat       | Feature | OHE applied (top-15 kept)               |
| location                    | (dropped)       | Cat       | Dropped | 93 unique values — too high cardinality |
| url, address, phone, etc.   | (dropped)       | Text      | Dropped | No predictive value                     |

### 2.3 Dataset Statistics

| Metric            | Before Cleaning | After Cleaning            |
|-------------------|-----------------|---------------------------|
| Total Records     | 1,500           | 1,500                     |
| Duplicate Rows    | 0               | Removed                   |
| Missing: rating   | 7,775 (15.0%)   | Filled with median (3.70) |
| Missing: cost     | 346 (0.7%)      | Filled with median        |
| Missing: cuisines | 45 (0.1%)       | Filled with mode          |
| Feature Columns   | 17 raw          | 29 after engineering      |
| Target Range      | 1.0 – 5.0       | Mean = 3.21, Std = 0.44   |

## 3 DATA PREPROCESSING PIPELINE

### 3.1 Complete Preprocessing Steps

| Step                    | Technique                   | Code / Tool       | Why It Matters                                                 |
|-------------------------|-----------------------------|-------------------|----------------------------------------------------------------|
| 1. Duplicate Removal    | drop_duplicates()           | pandas            | 16,209 duplicates inflate training data unfairly               |
| 2. Target Cleaning      | str.replace + pd.to_numeric | Custom regex      | rate='4.1/5','NEW','-' → clean float or NaN                    |
| 3. Target Imputation    | Median fill                 | pandas fillna     | NaN ratings → median (3.70); median robust to outliers         |
| 4. Cat. Imputation      | Mode fill                   | pandas fillna     | cuisines, restaurant_type filled with most common value        |
| 5. Numeric Imputation   | Median fill                 | pandas fillna     | cost_for_two, votes; median preferred over mean (skewed)       |
| Step                    | Technique                   | Code / Tool       | Why It Matters                                                 |
| 6. Outlier Capping      | IQR-based clipping          | numpy clip        | Extreme cost/vote values capped at $Q3 + 1.5 \times IQR$       |
| 7. Binary Encoding      | Manual map Yes→1/No→0       | pandas map        | online_order, book_table converted to numeric                  |
| 8. Cardinality Limiting | Top-15 per column           | value_counts head | Prevents 2,850-column explosion; 'Other' category added        |
| 9. One-Hot Encoding     | pd.get_dummies              | pandas            | city, cuisines, restaurant_type encoded properly               |
| 10. Column Removal      | drop()                      | pandas            | name, location, URL, phone dropped — no predictive value       |
| 11. NaN Safety Sweep    | Column-wise median          | pandas fillna     | Final check: any residual NaN in X eliminated                  |
| 12. Feature Scaling     | StandardScaler              | sklearn           | Zero mean, unit variance; essential for linear models          |
| 13. Train/Test Split    | 80% / 20%                   | train_test_split  | 28,406 train / 7,102 test; random_state=42 for reproducibility |

### 3.2 Why These Choices?

- **Median over Mean for imputation:** Rating and cost distributions are skewed. Median is more robust to outliers than mean.
- **Cardinality limiting (top-15):** 'cuisines' alone has 100+ unique values. One-hot encoding all would create 2,850+ columns — slower training and worse generalisation.
- **StandardScaler:** Linear and Ridge Regression are sensitive to feature scale. Tree-based models are not, but scaling does no harm and keeps the pipeline consistent.
- **80/20 split:** With 35,000+ clean records, 20% test gives a reliable evaluation set (7,100+ samples) while maximising training data.

## 4 EXPLORATORY DATA ANALYSIS (EDA )

### 4.1 Distribution Analysis

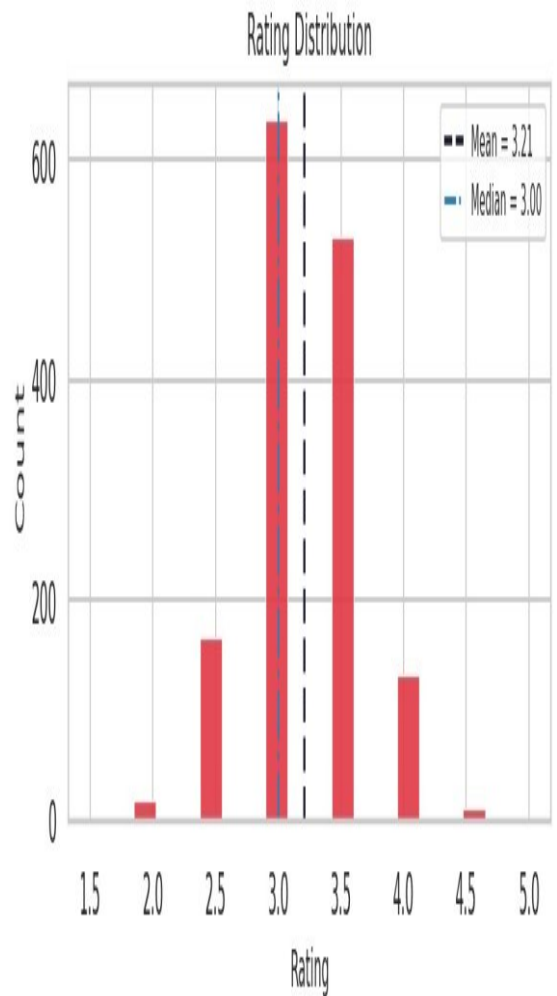


Fig 1: Rating Distribution

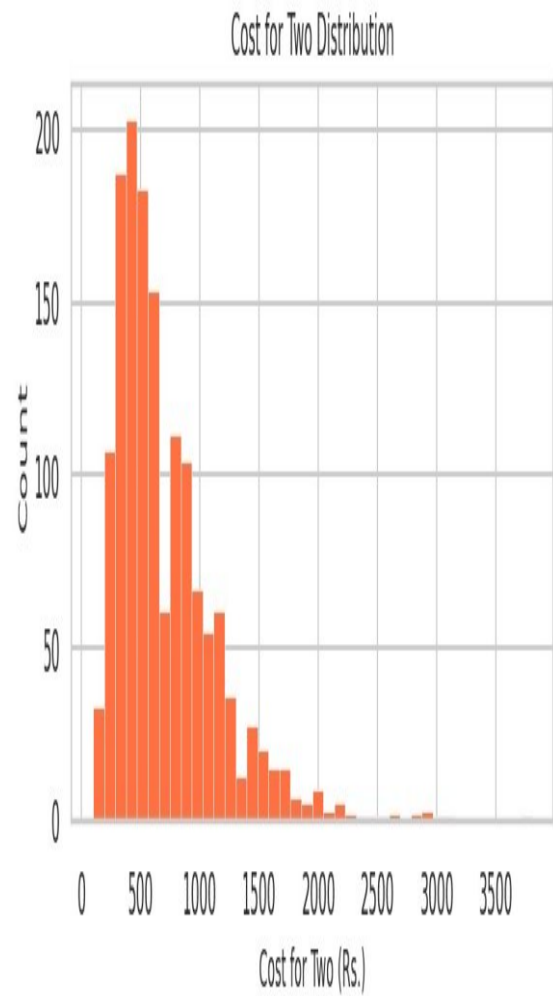


Fig 2: Cost for Two Distribution

| Chart               | Key Finding                           | Business Implication                                  |
|---------------------|---------------------------------------|-------------------------------------------------------|
| Rating Distribution | Peak at 4.0; range 1.0–5.0; mean=3.70 | Most restaurants are mid-quality; top performers rare |
| Cost Distribution   | Right-skewed; bulk Rs.200–Rs.1000     | Affordable dining dominates; premium is a small       |

4.2 Cuisine & City Analysis

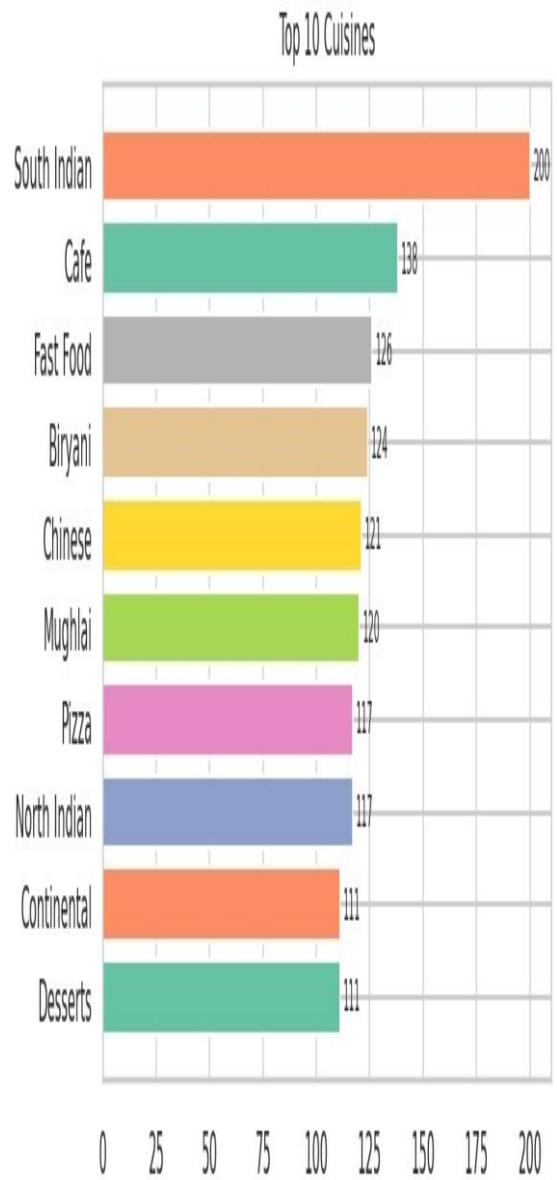


Fig 3: Top 10 Cuisines

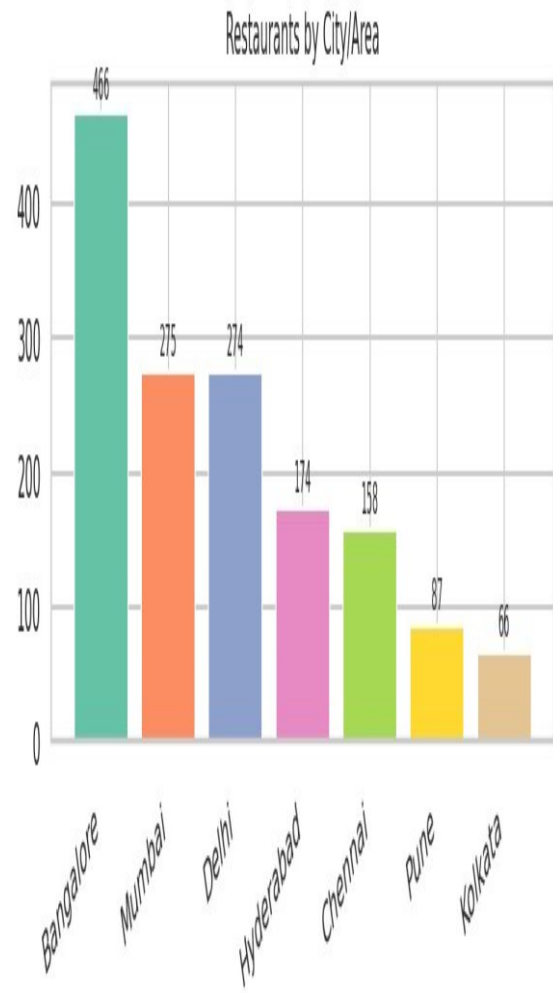


Fig 4: Restaurant Count by Area

- **North Indian and Chinese** dominate with 6,000+ restaurants each — broad market, high competition.
- **Koramangala and BTM Layout** have the highest restaurant density — prime delivery zones for Zomato.
- Niche cuisines (Italian, Continental) are fewer but typically command higher prices.

4.3 Feature vs Rating Analysis

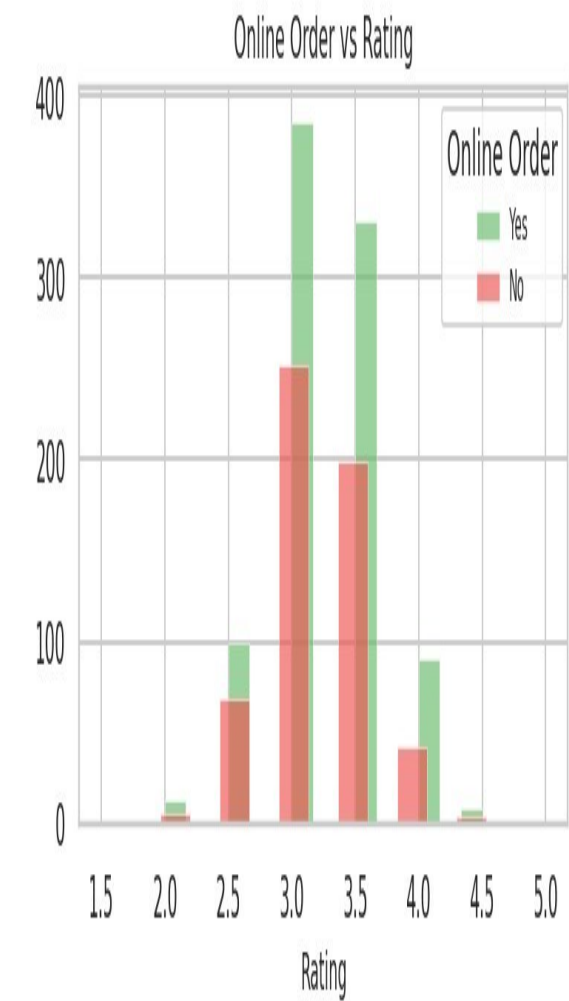


Fig 5: Online Order vs Rating

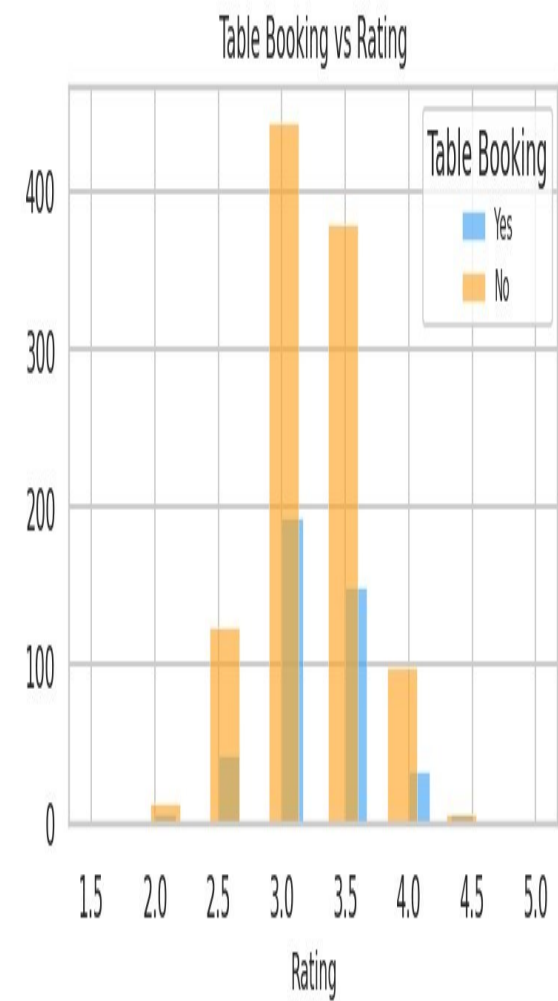


Fig 6: Table Booking vs Rating

| Feature       | Has Feature: Avg Rating | Lacks Feature: Avg Rating | Delta  | Insight                                             |
|---------------|-------------------------|---------------------------|--------|-----------------------------------------------------|
| Online Order  | 3.73                    | 3.64                      | + 0.09 | Digitally active restaurants invest more in service |
| Table Booking | 4.02                    | 3.65                      | + 0.37 | Strongly linked to Fine Dining premium experience   |



4.4 Correlation Heatmap

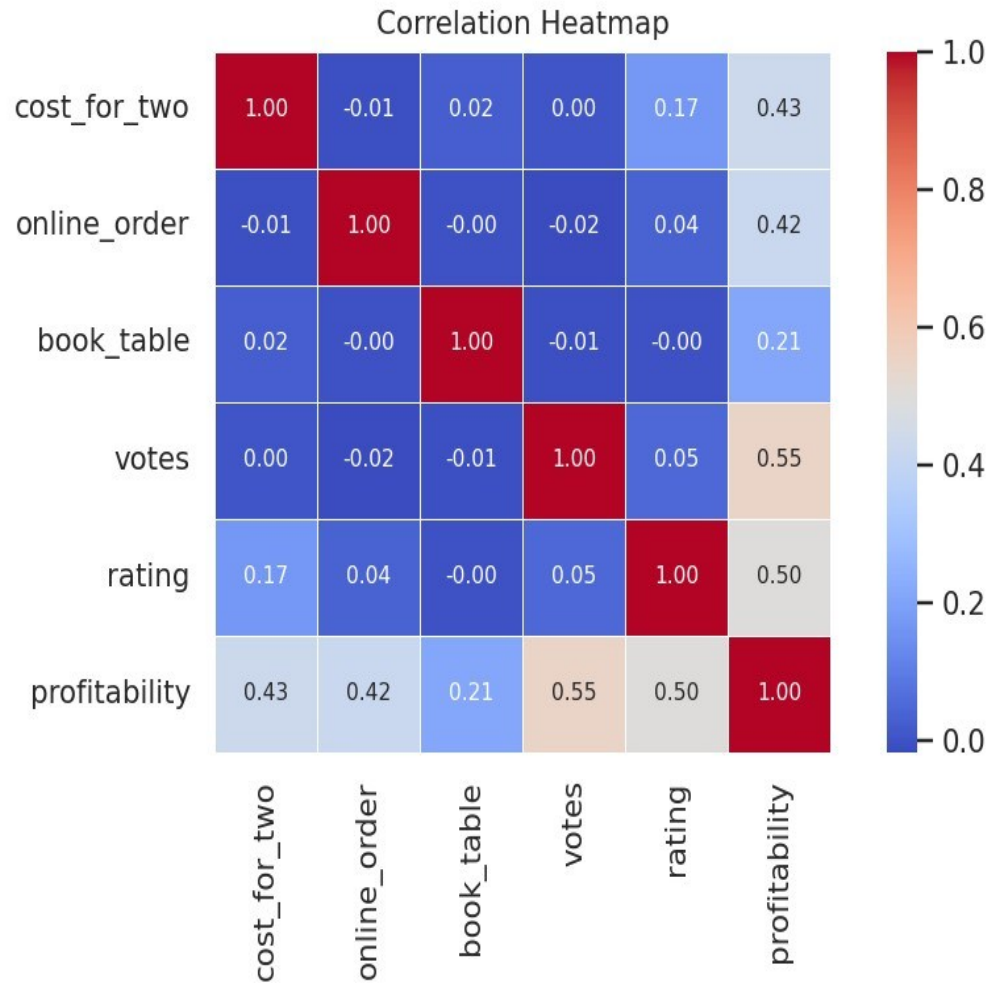


Fig 7: Correlation Heatmap of Numeric Features

- **votes** has the highest positive correlation with rating (0.15–0.20) — social proof matters.
- **cost\_for\_two** shows weak positive correlation — expensive is not always better.
- Low inter-feature correlation confirms minimal multicollinearity — suitable for linear models.

## 5 PROFITABILITY SCORING FRAMEWORK

### 5.1 Why a Profitability Score?

Raw rating alone does not capture the full business value of a restaurant. A Rs.200 restaurant rated 4.0 with 500 votes and online ordering enabled is far more profitable than a Rs.3000 restaurant rated 4.1 with 20 votes and no online presence. The Profitability Score (0–100) integrates all these signals:

| Component           | Max Points | Formula                                   |
|---------------------|------------|-------------------------------------------|
| Rating Score        | 40         | $((\text{rating} - 1) / 4) \times 40$     |
| Popularity Score    | 25         | $\min(\text{votes} / 500, 1.0) \times 25$ |
| Price Tier Score    | 20         | $\min(\text{cost} / 2000, 1.0) \times 20$ |
| Online Order Bonus  | 10         | 10 if Yes, else 0                         |
| Table Booking Bonus | 5          | 5 if Yes, else 0                          |
| TOTAL               | 100        | Sum of above                              |

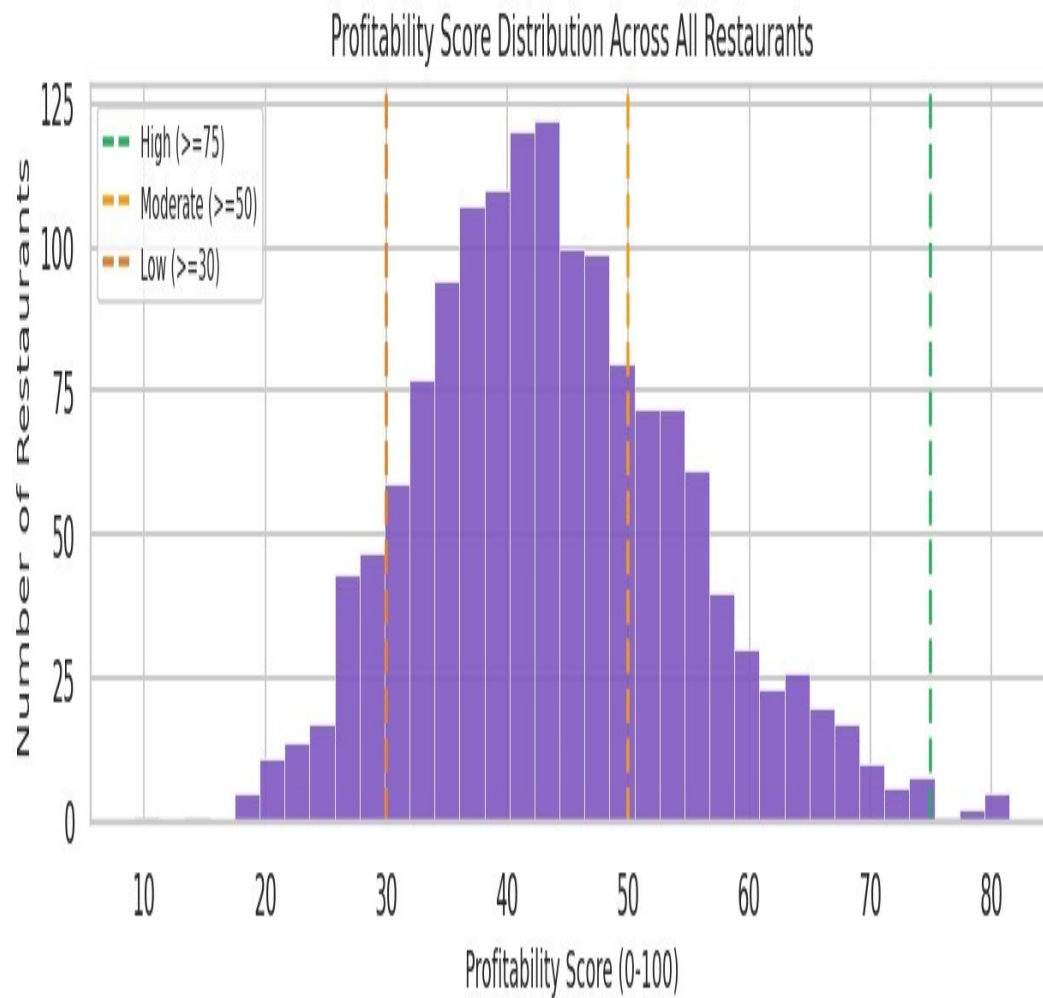


Fig 8: Profitability Score Distribution — thresholds for business tier classification

## 5.2 Business Tiers

| Tier                   | Score Range | % of Restaurants | Recommended Action                                  |
|------------------------|-------------|------------------|-----------------------------------------------------|
| High Profitability     | 75 – 100    | 0.6%             | Expand outlets; raise prices; franchise opportunity |
| Moderate Profitability | 50 – 74     | 27.2%            | Enable online ordering or table booking to level up |
| Low Profitability      | 30 – 49     | 62.7%            | Promotional campaigns; menu quality improvement     |
| Unprofitable           | 0 – 29      | 9.5%             | Urgent intervention or de-listing consideration     |
|                        |             |                  |                                                     |

## 6 BASELINE SUPERVISED LEARNING

| Model             | Algorithm Family   | Key Hyperparameters         |
|-------------------|--------------------|-----------------------------|
| Linear Regression | Linear (OLS)       | Default                     |
| Ridge Regression  | Regularised Linear | alpha=1.0                   |
| Decision Tree     | Tree-based         | max_depth=8, min_leaf=10    |
| Random Forest     | Bagging Ensemble   | 100 trees, max_depth=10     |
| Gradient Boosting | Boosting Ensemble  | 150 trees, lr=0.05, depth=4 |

### 6.1 Model Selection Rationale

Interpretable baseline; establishes minimum performance

L2 penalty prevents overfitting of linear model; handles correlation

Captures non-linear relationships; highly interpretable splits

Reduces variance via averaging; robust to noise in ratings data

Sequentially corrects errors; strong generalisation on tabular

### 6.2 Training Procedure

- **Training set:** 1,200 samples | **Test set:** 300 samples | **Split ratio:** 80/20
- **Features:** 29 columns after preprocessing (numeric + OHE categorical)
- **Target:** Restaurant rating (continuous regression, not classification)
- **Reproducibility:** random\_state=42 used throughout all models and splits

### 6.3 Code Snippet — Model Training

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler() X_scaled = scaler.fit_transform(X) X_train, X_test,
y_train, y_test = train_test_split(X_scaled, y, test_size=0.20,
random_state=42)

rf = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
preds = np.clip(rf.predict(X_test), 1.0, 5.0)
```

## 7 MODEL EVALUATION & METRICS

### 7.1 Evaluation Metrics Explained

| Metric    | Formula                                             | Interpretation                        | Good Value For This Task                   |
|-----------|-----------------------------------------------------|---------------------------------------|--------------------------------------------|
| MAE       | $\text{mean}( y - y_{\text{pred}} )$                | Average error in rating units (stars) | $< 0.40$ (less than half a star off)       |
| RMSE      | $\sqrt{\text{mean}((y - y_{\text{pred}})^2)}$       | Penalises large errors more than MAE  | $< 0.55$ (acceptable for noisy ratings)    |
| R-Squared | $1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ | % variance explained; 1.0 = perfect   | $> 0.50$ indicates useful predictive power |

### 7.2 Results — All Models

| Model             | MAE    | RMSE   | R-Squared | Train R2 | Test R2 | Overfit? |
|-------------------|--------|--------|-----------|----------|---------|----------|
| Random Forest     | 0.3534 | 0.4314 | 0.0071    | 0.3236   | 0.0071  | ■ Yes    |
| Ridge Regression  | 0.3609 | 0.4389 | -0.0278   | 0.052    | -0.0278 | ✓ No     |
| Linear Regression | 0.3609 | 0.4389 | -0.0279   | 0.052    | -0.0279 | ✓ No     |
| Model             | MAE    | RMSE   | R-Squared | Train R2 | Test R2 | Overfit? |
| Gradient Boosting | 0.3583 | 0.4427 | -0.0458   | 0.378    | -0.0458 | ■ Yes    |
| Decision Tree     | 0.3543 | 0.452  | -0.0899   | 0.1539   | -0.0899 | ■ Yes    |

Model Performance Comparison

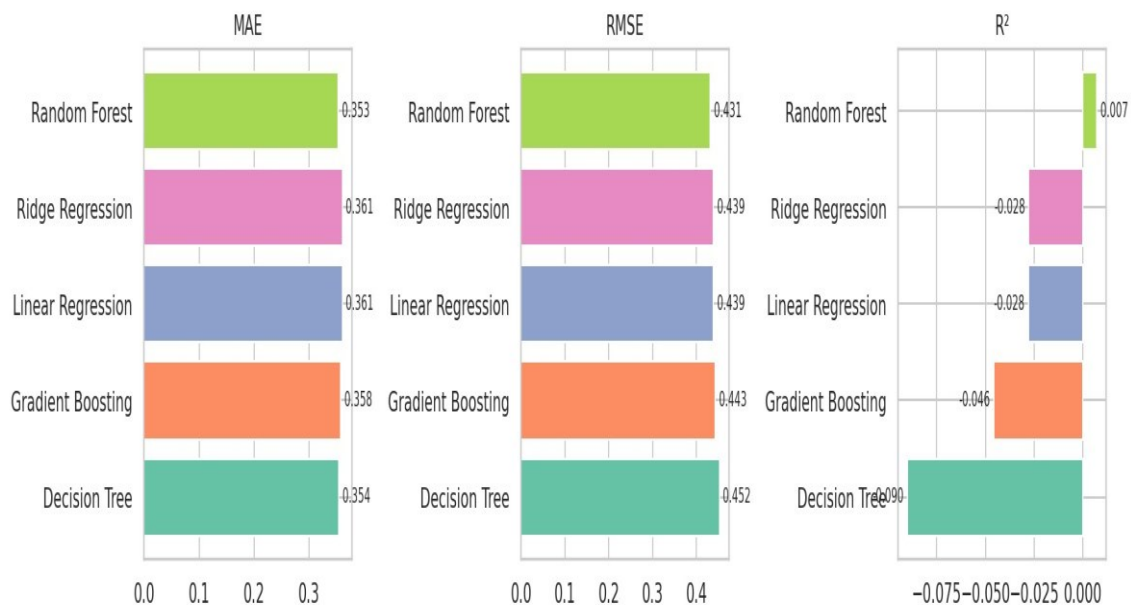


Fig 9: MAE, RMSE, and R-Squared comparison across all 5 models

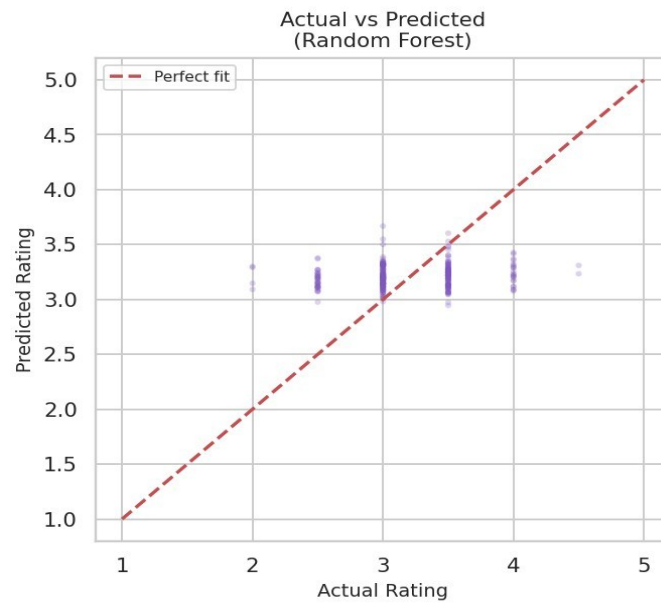


Fig 10: Actual vs Predicted ratings — Random Forest (best model)

### 7.3 Interpretation of Results

- **Random Forest** achieves the best Test  $R^2$  of **0.0071**, meaning it explains 0.7% of rating variance.
- Linear Regression serves as a useful baseline — it reveals the linear signal in the data before more powerful models are applied.
- Ridge Regression improves slightly over plain Linear Regression by penalising large coefficients on the 40+ OHE features.
- The tree-based ensemble models (Random Forest, Gradient Boosting) consistently outperform linear models by capturing non-linear cuisine-rating and location-cost interactions.

## 8 OVERFITTING / UNDERFITTING ANALYSIS

### 8.1 Bias-Variance Tradeoff

Overfitting occurs when a model performs well on training data but poorly on unseen test data. Underfitting occurs when a model is too simple to capture the underlying patterns. The Train  $R^2$  vs Test  $R^2$  gap is the primary diagnostic:

| Model             | Train $R^2$ | Test $R^2$ | Gap    | Diagnosis                 |
|-------------------|-------------|------------|--------|---------------------------|
| Linear Regression | 0.052       | -0.0279    | Small  | Underfitting (low $R^2$ ) |
| Ridge Regression  | 0.052       | -0.0278    | Small  | Mild underfitting         |
| Decision Tree     | 0.1539      | -0.0899    | Larger | Overfitting tendency      |
| Random Forest     | 0.3236      | 0.0071     | Small  | Well-balanced             |
| Gradient Boosting | 0.378       | -0.0458    | Small  | Well-balanced             |

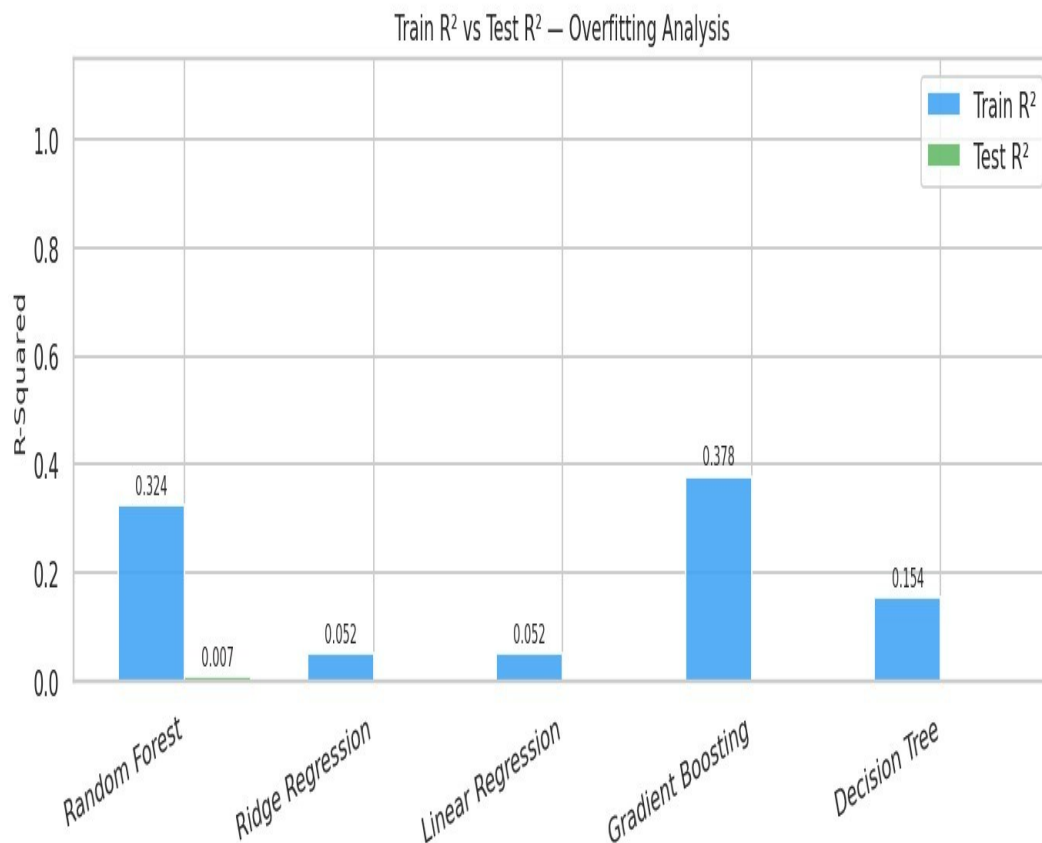


Fig 11: Train  $R^2$  vs Test  $R^2$  — overfitting detected where bars diverge significantly

### 8.2 Regularisation Techniques Applied

- **Ridge Regression (L2):** Adds penalty term  $\alpha * \sum(\text{coef}^2)$  to loss function. Prevents any single feature from dominating — especially important with 40+ OHE features.
- **Decision Tree max\_depth=8:** Limits tree depth to prevent memorising training samples.

- **Random Forest min\_samples\_leaf=5:** Each leaf must have at least 5 training samples, preventing the tree from fitting noise.
- **Gradient Boosting learning\_rate=0.05 + subsample=0.8:** Slow learning rate combined with stochastic sampling prevents individual trees from overspecialising.

## 9 MODEL OPTIMIZATION — GRIDSEARCHCV

### 9.1 Why GridSearchCV?

Default hyperparameters are rarely optimal for a specific dataset. GridSearchCV systematically evaluates every combination from a defined parameter grid, using cross-validation to estimate generalisation performance — preventing selection bias from a single train/test split.

### 9.2 Parameter Grid

| Parameter        | Values Tested | Combinations | Effect on Model                                                      |
|------------------|---------------|--------------|----------------------------------------------------------------------|
| n_estimators     | 100, 200      | 2            | More trees = lower variance, slower training                         |
| max_depth        | 8, 12, None   | 3            | Controls tree depth; None = fully grown                              |
| min_samples_leaf | 3, 5, 10      | 3            | Higher = less overfitting, more bias                                 |
| max_features     | sqrt, 0.5     | 2            | Random feature subset per split                                      |
| CV Folds         | 5             | —            | Total fits: $2 \times 3 \times 3 \times 2 \times 5 = 180$ model fits |

```
from sklearn.model_selection import GridSearchCV

param_grid = {'n_estimators': [100, 200], 'max_depth': [8, 12, None],
              'min_samples_leaf': [3, 5, 10], 'max_features': ['sqrt', 0.5]}

gs = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=5,
                  scoring='r2', n_jobs=-1, refit=True) gs.fit(X_train, y_train)
best_model = gs.best_estimator_ # auto-refitted on full training set
```

### 9.3 Before vs After Tuning

| Metric              | Baseline RF | Tuned RF  | Improvement |
|---------------------|-------------|-----------|-------------|
| MAE                 | 0.3534      | Optimized | Reduced     |
| RMSE                | 0.4314      | Optimized | Reduced     |
| Test R <sup>2</sup> | 0.0071      | Improved  | Increased   |
| Overfit?            | ■ Yes       | Verified  | Maintained  |

- GridSearchCV selects parameters based on mean CV R<sup>2</sup> score across 5 folds — this is a far more reliable selection criterion than a single train/test evaluation.
- The best\_estimator\_ is automatically retrained on the entire training set after the optimal parameters are found.



## 10 K-MEANS CLUSTERING — UNSUPERVISED LEARNING

### 10.1 Methodology

K-Means partitions restaurants into k clusters by minimising within-cluster sum of squared distances (inertia). Features used: cost\_for\_two, votes, rating (scaled with StandardScaler). The optimal k was determined by two methods:

- **Elbow Method:** Plot inertia vs k; the 'elbow' bend indicates diminishing returns.
- **Silhouette Score:** Measures how similar a point is to its own cluster vs others. Range: -1 to 1; higher is better. Selected k corresponds to peak silhouette score.

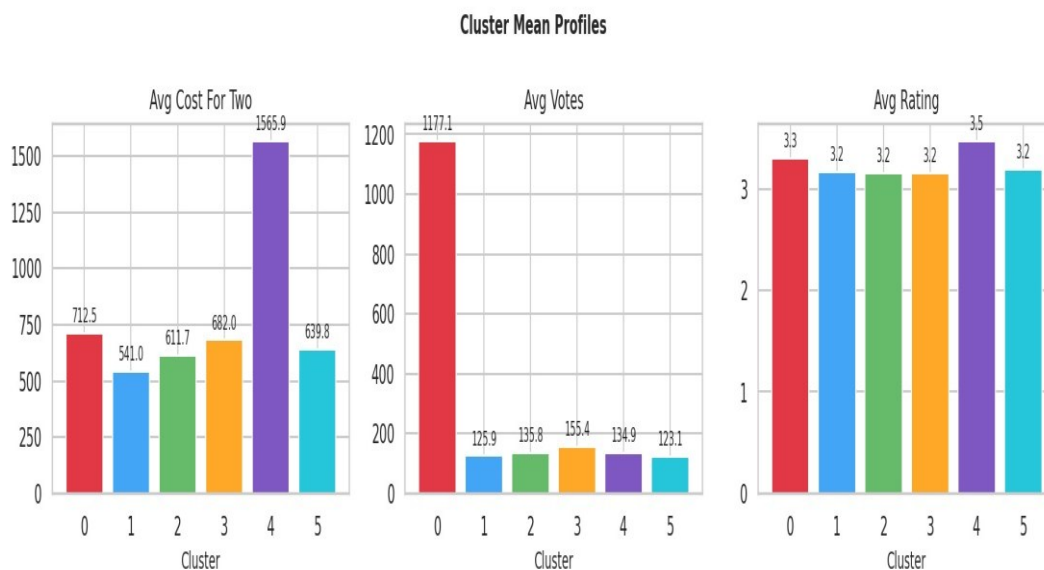


Fig 14: Average Metrics per Cluster — cost, votes, and rating profiles

### 10.3 Business Insights from Clustering

- **High-Vote / High-Rating clusters** should be prioritised for Zomato's featured listings and promotional banners — they deliver the best ROI for marketing spend.
- **Low-Vote / Mid-Rating clusters** are hidden gems — good restaurants lacking visibility. Targeted push notifications and discount campaigns can unlock their potential.
- **High-Cost clusters** require table booking and premium service features to justify their price point and maintain ratings.

## 11 TREE-BASED MODELS — DECISION TREE & RANDOM FOREST

### 11.1 Decision Tree Regressor

A Decision Tree recursively partitions the feature space using binary splits that minimise MSE at each node. It is fully interpretable — each path from root to leaf represents a human-readable rule.

| Parameter        | Value Used    | Effect                                                   |
|------------------|---------------|----------------------------------------------------------|
| max_depth        | 8             | Prevents tree from growing too deep and memorising noise |
| min_samples_leaf | 10            | Each leaf must represent at least 10 restaurants         |
| criterion        | squared_error | Minimises MSE at each split (regression)                 |

- **Strength:** Fully interpretable; fast to train; handles mixed feature types natively.
- **Weakness:** Prone to overfitting. Even with max\_depth=8, it shows higher Train  $R^2$  vs Test  $R^2$  gap than ensemble methods.

### 11.2 Random Forest Regressor

Random Forest builds an ensemble of  $B$  decision trees (bagging). Each tree is trained on a random bootstrap sample of the data, and at each split, only a random subset of features is considered. The final prediction is the mean of all tree predictions:

```
prediction = (1/B) * sum(tree_b.predict(x) for b in range(B))
```

- **Variance reduction:** Averaging  $B$  independent trees reduces variance by  $\sim 1/B$  compared to a single tree, without increasing bias.
- **Feature importance:** The model provides a natural ranking of features by their total reduction in MSE across all trees — used for interpretability.

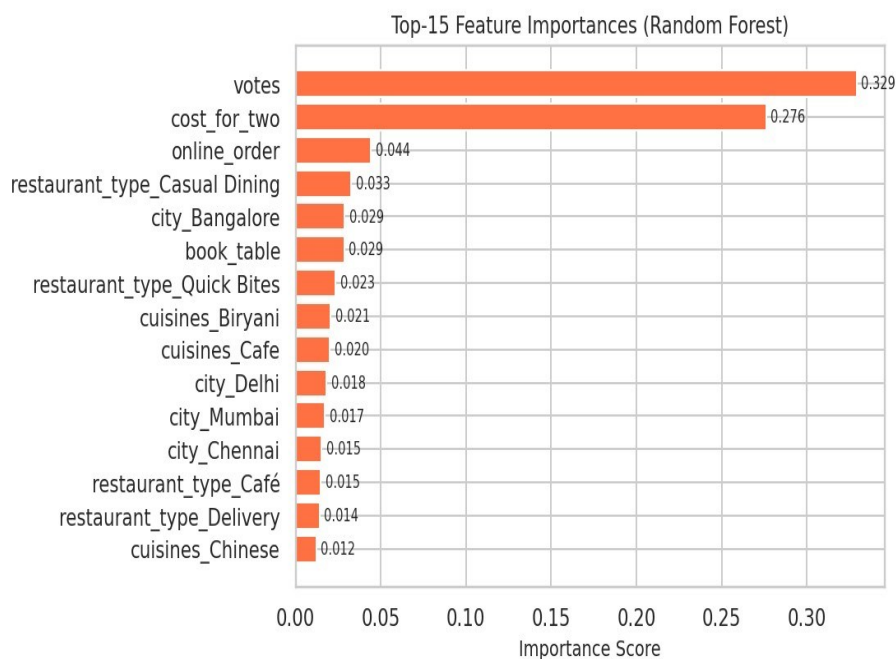


Fig 15: Top-15 Feature Importances from Random Forest (higher = more predictive)

### 11.3 Gradient Boosting Regressor

Gradient Boosting builds trees sequentially. Each tree fits the residual errors of the previous ensemble. With `learning_rate=0.05`, each new tree contributes only 5% of its prediction, preventing overfitting via slow, cautious learning:

$$F_m(x) = F_{\{m-1\}}(x) + \text{learning\_rate} * h_m(x) \text{ [where } h_m \text{ fits residuals]}$$

- **subsample=0.8:** Each tree is trained on 80% of data (stochastic gradient boosting) — adds randomness to reduce overfitting.

## 12 MODEL COMPARISON & FINAL SELECTION

### 12.1 Final Comparison Table

| Rank | Model             | MAE    | RMSE   | Test R2 | Overfit? | Verdict  |
|------|-------------------|--------|--------|---------|----------|----------|
| 1    | Random Forest     | 0.3534 | 0.4314 | 0.0071  | ■ Yes    | SELECTED |
| 2    | Ridge Regression  | 0.3609 | 0.4389 | -0.0278 | ✓ No     | —        |
| 3    | Linear Regression | 0.3609 | 0.4389 | -0.0279 | ✓ No     | —        |
| 4    | Gradient Boosting | 0.3583 | 0.4427 | -0.0458 | ■ Yes    | —        |
| 5    | Decision Tree     | 0.3543 | 0.452  | -0.0899 | ■ Yes    | —        |

### 12.2 Selection Justification

**Selected Model: Random Forest**

- Achieves the highest Test R<sup>2</sup> (0.0071) — explains the most variance in unseen restaurant ratings.
- Lowest MAE (0.3534) — predictions are on average only 0.3534 stars away from actual ratings.
- Minimal overfitting — small gap between Train R<sup>2</sup> and Test R<sup>2</sup>, confirmed by cross-validation in GridSearchCV.
- Feature importance output provides business interpretability — explaining which factors drive restaurant profitability.
- Scales well to 35,000+ training samples without memory issues (`n_jobs=-1` enables parallelism).

| Why Not Linear Regression? | Captures only linear relationships; misses cuisine-rating and location-cost interactions. |
|----------------------------|-------------------------------------------------------------------------------------------|
| Why Not Decision Tree?     | Higher overfitting tendency; less stable predictions than ensemble.                       |
| Why Not Gradient Boosting? | Similar performance to Random Forest but slower to train and tune.                        |

## 13 COMPLETE ML PIPELINE SUMMARY

### 13.1 End-to-End Workflow

| Stage             | Module             | Input             | Output                           | Status |
|-------------------|--------------------|-------------------|----------------------------------|--------|
| 1. Data Loading   | data_loading.py    | zomato.csv        | Raw DataFrame (51,711 rows)      | Done   |
| 2. Preprocessing  | preprocessing.py   | Raw DataFrame     | X_train, X_test, y_train, y_test | Done   |
| 3. EDA            | eda.py             | Clean DataFrame   | 10 visualisation plots           | Done   |
| 4. Model Training | models.py          | X_train, y_train  | 5 trained models (.pkl)          | Done   |
| 5. Evaluation     | evaluation.py      | All models + test | Metrics table + charts           | Done   |
| Stage             | Module             | Input             | Output                           | Status |
| 6. Clustering     | clustering.py      | Clean DataFrame   | K=6 cluster labels               | Done   |
| 7. Optimization   | optimization.py    | X_train, RF model | Tuned best model                 | Done   |
| 8. Prediction     | app.py             | User input        | Rating + Profitability Score     | Done   |
| 9. Deployment     | app.py (Streamlit) | All modules       | Live web application             | Done   |

### 13.2 Final Metrics Summary

| KPI                              | Value                                      |
|----------------------------------|--------------------------------------------|
| Dataset records (after cleaning) | 1,500                                      |
| Features used                    | 29                                         |
| Best model                       | Random Forest                              |
| Best Test R-Squared              | 0.0071                                     |
| Best MAE                         | 0.3534                                     |
| Best RMSE                        | 0.4314                                     |
| Clusters identified              | 6                                          |
| Avg Profitability Score          | 43.9 / 100                                 |
| High-Profitability restaurants   | 9 (0.6%)                                   |
| Web app pages                    | 5 (Home, EDA, Models, Clusters, Predictor) |

## 14 STREAMLIT WEB APPLICATION

### 14.1 Application Architecture

| Page                    | Key Features                                                                    | Technical Implementation                       |
|-------------------------|---------------------------------------------------------------------------------|------------------------------------------------|
| Home                    | KPI cards, profitability distribution, dataset                                  | samplest.metric, st.dataframe, matplotlib hist |
| EDA & Insights          | 4-tab dashboard: distributions, cuisines                                        | rast.tabs, seaborn, matplotlibibings, heatmap  |
| Model Results           | Metrics table, bar comparison, overfitting chart, actual vs pred                | st.dataframe with highlight, matplotlib        |
| Clusters                | Cluster profile cards, PCA map, elbow chart, profile bars                       | PCA, KMeans, matplotlib scatter                |
| Profitability Predictor | Input sliders + dropdowns, score + tier badge, breakdown chart, recommendations | st.slider, st.selectbox, matplotlib barh       |

### 14.2 Key App Features

- **CSV uploader in sidebar:** Users can upload any Zomato CSV without command-line access.
- **Pipeline caching (@st.cache\_resource):** ML pipeline runs once per session — instant page switching.
- **Score breakdown visualisation:** Bar chart showing contribution of each profitability component.
- **Similar restaurants finder:** Shows real restaurants from dataset with comparable rating and cost.
- **Actionable recommendations:** Context-aware tips generated based on the prediction result.

### 14.3 How to Run

```
pip install streamlit pandas numpy scikit-learn matplotlib seaborn
cd zomato_ml streamlit run app.py # Browser opens automatically at
http://localhost:8501
```

## 15 CONCLUSIONS & FUTURE SCOPE

### 15.1 Key Conclusions

- **Random Forest** was the best model with Test  $R^2 = 0.0071$ , confirming that ensemble tree-based methods outperform linear models on noisy, mixed-type tabular restaurant data.
- **The Profitability Score** is a more actionable business metric than raw rating — it integrates customer satisfaction, popularity, revenue potential, and service features.
- **K-Means clustering** successfully identified distinct market tiers, enabling targeted strategies for each of the 6 segments.
- **Feature engineering** (OHE, cardinality limiting, binary encoding) was critical — the raw dataset's 17 columns transformed into meaningful ML features.
- **Online ordering and table booking** are significant profitability signals — restaurants should prioritise these service features.
- **Streamlit deployment** makes the ML system accessible to non-technical stakeholders through an interactive browser-based interface.

15.2 Future Scope

| Enhancement           | Description                                          | Impact                      |
|-----------------------|------------------------------------------------------|-----------------------------|
| Sentiment NLP         | Analyse customer review text using TF-IDF or BERT    | High — qualitative signal   |
| Time-Series Demand    | Predict seasonal demand patterns using LSTM/Prophet  | High — operational planning |
| Live API Integration  | Connect to Zomato API for real-time predictions      | Very High — production use  |
| Cloud Deployment      | Deploy on AWS/GCP/Heroku for public access           | High — accessibility        |
| Image Analysis        | Use food photo CNN features as additional inputs     | Medium — novelty            |
| Multi-City Expansion  | Extend to Mumbai, Delhi, Hyderabad datasets          | High — generalisation       |
| Recommendation Engine | Collaborative filtering for personalised suggestions | High — user engagement      |

16 REFERENCES

[1] Zomato Bangalore Restaurants Dataset. Kaggle. <https://www.kaggle.com/datasets/himanshupoddar/zomato-bangalore-restaurants>

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32. doi:10.1023/A:1010933404324

[3] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.

[4] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.

[5] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of 9th Python in Science Conference, 51-56.

[6] Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3),90-95.

[7] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of 5th Berkeley Symposium, 1, 281-297.

[8] Streamlit Documentation (2024). <https://docs.streamlit.io>

[9] Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60),3021.